

PA 3,4 - Lists, Stacks & Queues - Refining a Shortlist of Candidate Meals

Units 3 & 4: implement a circular linked list, and use lists, stacks, and queues. These files come from AI/statistics problems, but you need none of that background — everything you need is right here.

This assignment continues the **Menu Planner**. A customer's **shortlist of candidate meals** starts **empty** and grows as **requests** arrive — **add a meal**, **set a course** (fix one course to one dish), or **remove the lowest** few. You keep the active shortlist in a **circular linked list** you build (by adapting a plain linked list we give you). Requests come in on a **queue** and you apply them in arrival order; each *successful* request can be **undone** — **undo** the most recent with a **stack**, where reversing an add-meal, a set-course, or a remove-lowest each does something different. Diners always ask about the top pick, so the **first prioritized meal** (the highest-scoring) should stay easy to reach. Finally you **rotate** through the finished shortlist. Throughout, you **move** meals between lists rather than copying them. The point spans **Units 3 & 4**: implementing a ring, and using each structure for what it is good at.

At a Glance

- **Submit on Gradescope:** two files — `CircularSinglyLinkedList.hpp` and `Shortlist.hpp`.
- **Given (do not modify):** the whole model, a *complete* plain `SinglyLinkedList`, `Stack`, `Queue`, `Request`, and `mealScore` — each in its own header.
- **You implement:** `CircularSinglyLinkedList` — adapt the plain list into a ring (marked `// TODOs`).
- **You design:** the `Shortlist` **class** — its six operations *use* these structures, and you choose how to store its heterogeneous **undo history**.
- **Graded on:** memory safety first (a leak scores 0), your circular list's behavior (conformance), your `Shortlist`'s request behavior and its methods' *results* (order-independent), that you **move** meals rather than copy them, and a small **performance** check.

THE PROJECT FILES

Each class lives in its own header. **Two** are yours to edit and submit; the rest are provided complete — read them, especially the plain `SinglyLinkedList`, which is the template you adapt.

File	Role
<code>Course.hpp</code> , <code>ChosenCourse.hpp</code> , <code>PairingFactor.hpp</code> , <code>MenuModel.hpp</code> , <code>Meal.hpp</code> , <code>mealScore.hpp</code>	the menu model (provided)
<code>SinglyLinkedListNode.hpp</code>	one node, shared by BOTH lists (provided)
<code>SinglyLinkedList.hpp</code>	a plain singly-linked list, complete — your template (provided)
<code>Stack.hpp</code> , <code>Queue.hpp</code> , <code>Request.hpp</code>	LIFO / FIFO / a request the queue carries (provided)
<code>CircularSinglyLinkedList.hpp</code>	YOU IMPLEMENT the ring (submit)
<code>Shortlist.hpp</code>	YOU DESIGN the <code>Shortlist</code> class (submit)
<code>driver.cpp</code>	local test harness (provided; do not modify)

The active shortlist is a `CircularSinglyLinkedList<Meal>`. Each successful request must be undoable, so you keep an **undo history** — and because reversing an *add-meal*, a *set-course*, or a *remove-lowest* each does something different, one history entry must remember enough to undo *one request of any kind*. **How you store that history is yours to design**. When a request moves meals out, a set-aside list can be a plain `SinglyLinkedList<Meal>`, which shares the same `SinglyLinkedListNode<Meal>` as the ring — exactly what lets you **move** a meal from one list into the other by relinking one node, *no copy*.

</> PART 1 — IMPLEMENT THE CIRCULAR LIST

Open `CircularSinglyLinkedList.hpp`. It is a copy of the plain `SinglyLinkedList` scaffolding with the ring mechanics left as `// TODO`. A **ring** keeps `_rear->next == front` (there is no `nullptr` tail), so two things differ from the plain list:

- **The ends are found through `_rear`**: the **front** is `_rear->next`, the **back** is `_rear`. Insertions must keep the ring closed.
- **The cursor is count-bounded**: a ring has no null to stop at, so `atEnd()` compares the cursor's index to the list's *current* length (which shrinks as you detach).

You fill in: `front/back`, `rotate` (advance `_rear` one hop, $O(1)$), `append/prepend` (keep the ring closed), the node-level `detachAt / attachBack / attachBefore` (unlink / relink an *existing* node — no copy), and the cursor's `atEnd()`. The plain `SinglyLinkedList` shows you each one for the non-circular case; keep the ring closed.

≡ PART 2 — DESIGN THE SHORTLIST

Open `Shortlist.hpp`. A `Shortlist` starts **empty** and owns its active list, its undo history, and any book-keeping it allocates. The active list is a `CircularSinglyLinkedList<Meal>` (given). **You design** two private things: (a) the **undo history** — one entry must remember enough for `undoLast` to reverse *one request of any kind*, so make `_undo` a `Stack` of that record; and (b) how to **track which courses are set**. **You also choose** how to order the active list; correctness does not depend on it — and the measured checks reward a wise choice. One more wish from the diners: they always ask about the top pick, so the **first prioritized meal** — the **highest**-scoring one — should be easy to reach at any moment, in $O(1)$. The read-only view (`size`, `isEmpty`, `undoDepth`, `begin`, `end`) is given — keep it, and keep the six operations below (the autograder calls them).

1. `addMeal(meal, model)` — add meal in score order and return `true`, *unless* it disagrees with a course that has been set, in which case reject it and return `false`. Walk a cursor to the sorted spot, then attach the meal's node. Use `mealScore(model, meal)`.
2. `setCourse(course, dish, model)` — if course is *already* set, do nothing and return `false`. Otherwise record it, **move** out every meal that picks a different dish for course (keep them for undo), block future add-meals that disagree, and return `true`.
3. `removeLowest(k, model)` — **move** the `k` lowest-scoring meals out and keep them for undo. If `k` is more than the current size, remove **all** of them.
4. `applyRequests(requests, model)` — take each request off the **queue** in arrival order and dispatch it by kind to `addMeal / setCourse / removeLowest`.
5. `undoLast(model)` — reverse the **most recent successful** request by popping your undo **stack** and dispatching on the record's kind: an *add-meal* — find that meal, remove it, and `delete` it (which **destroys** the meal; there is no redo); a *set-course* — unset that course and move its removed meals back in score

order; a *remove-lowest* — move its removed meals back. Free anything the record owned. With an **empty** undo history it does nothing.

6. `featuredMeal()` and `rotate()` — the finale, this time driven by the caller. `featuredMeal()` returns a `Meal const&` for the meal currently in the spotlight (whichever end of the ring your rotation exposes), so it copies *nothing*; `rotate()` advances the ring one hop in $O(1)$. Calling them in turn for `size()` turns must visit every meal exactly once. Like the ring's `front()`, `featuredMeal()` may assume the shortlist is not empty.

You own the undo history — and its shape is your call. One record must be able to reverse *any* kind of request: an *add-meal* (which meal to find and destroy), a *set-course* (which course to unset *and* the meals it removed), or a *remove-lowest* (the meals it removed). In the kitchen, removed meals wait on a **numbered tray**, each pointed to by its **position**, until they are restored or discarded. Store your set-aside meals however you like, and track *which courses are fixed* however you like (an array, a list, ...) — you are graded on the **outcome**, not your representation. Whatever the `Shortlist` sets aside it **owns**: free every still-pending set-aside list *and* any bookkeeping you allocated (destructor + rule of five). The memory gate fails the whole assignment on any leak.

Two behaviors to get exactly right. (1) `removeLowest` with k larger than the current number of active meals removes **all** of them — this is the **defined behavior, not an error**. (2) **A request that changes nothing records nothing**: a rejected add-meal, a set-course on a course that is already set, and a remove-lowest that removes nothing ($k \leq 0$, or an empty shortlist) leave the undo history untouched, so `undoLast` only ever reverses a request that actually changed the shortlist.

ANALYZE YOUR CODE

Be ready to state each method's **Big-O** and *why*, in N (meals in the list) and k (meals a request removes). Where a bound below spans a range, *your design decides* which end of the range you get — working out how is part of the assignment:

Method	Time
<code>addMeal</code>	$O(N)$ — find the sorted spot, then an $O(1)$ splice
<code>setCourse</code>	$O(N)$ — one pass; each node move is $O(1)$
<code>removeLowest</code>	$O(k)$ to $O(N \cdot k)$, depending on your design ($k > \text{size} \Rightarrow \text{all}$)
<code>applyRequests</code>	sum of its requests' costs
<code>undoLast</code>	an add-meal: $O(N)$ to find + destroy; a set-course / remove-lowest: up to $O(k \cdot N)$ re-inserting one by one — how much of that work is <i>truly</i> needed depends on your undo design
<code>featuredMeal / rotate</code>	$O(1)$ each — so a full lap of <code>size()</code> turns is $O(N)$

Why a ring, and why move not copy? A meal owns a heap array, so *copying* one is real work: it allocates a new array and deep-copies every course — and transferring a meal that way then has to destroy the original. **Moving** the node instead is a single pointer relink — $O(1)$, with no allocation and no copy. And because the same meals are removed and reinserted over and over (a set-course, a remove-lowest, then an undo), that saved work adds up quickly. `rotate()` likewise shifts the whole ring

one hop in $O(1)$ once the shortlist is final.

RULES

- **Submit only** `CircularSinglyLinkedList.hpp` and `Shortlist.hpp`; do not modify the other files.
- **Standard library for small utilities only** — in your two files you may include *only* `<utility>`, `<string>`, `<fstream>`, `<stdexcept>` (the provided scaffolding already includes `<utility>` for `std::swap`). **No other standard header**, including `<iostream>` — take debug prints out before you submit. **Do not reimplement or swap in** `std::list` / `std::stack` / `std::queue` / `std::vector` or any other STL container or smart pointer — you build the data structures. **Do not call the library sort / search / merge algorithms** either (`std::sort`, `std::merge`, `std::lower_bound`, ...). (The autograder enforces all of this; a banned construct scores 0 for the whole assignment, and naming the instructor's counters — `scoreCallCount`, `mealCopyCount` — does too. Your own helper classes are fine, except in the ring file; see the next rule.)
- **Build the ring on the provided node.** In `CircularSinglyLinkedList.hpp` you may define *only* the class itself and its provided nested `Cursor` and `Iterator` — do *not* declare a node type of your own. The exercise is a **singly**-linked ring, and a hand-rolled node carrying a `prev` pointer is a doubly-linked list. (The autograder enforces this; a violation scores 0.) Adding helper *methods* to your ring is fine — the ban is on new *types*. `Shortlist.hpp` is *not* restricted — the undo record you design there is your own.
- **No** using namespace — qualify names explicitly.
- **Move meals, do not copy them — this is graded (8 pts).** In `setCourse`, `removeLowest`, and `undoLast`, transfer a meal by detaching its **node** from one list and attaching that *same* node into the other — never by copying a `Meal`. (Undoing an add-meal is the one exception: it deletes the node, *destroying* that meal — still not a copy.) Read meals through `Meal const&` (a cursor's `*c`); do *not* assign one into a local `Meal` variable, which copies it. (`addMeal` *does* copy a meal by design and is not counted; `featuredMeal` hands back a reference, so it copies nothing at all.) The driver prints how many meals you copied, so you can check before submitting.
- **No leaks, no double-frees.** Keep the ring closed; every new needs a matching `delete`. The memory gate checks this.

BUILD & TEST LOCALLY

```
cd "PA 3,4 - Lists, Stacks & Queues - Refining a Shortlist of Candidate Meals/starter"
g++ -std=c++20 -O2 -Wall -Wextra driver.cpp -o refine
./refine samples/menu3.menu samples/menu3.chosen
diff <(. /refine samples/menu3.menu samples/menu3.chosen) samples/expected_menu3.txt #
prints nothing if correct
```

The provided `driver.cpp` grows a small shortlist from empty, runs your methods (a few add-meals, a set-course, a remove-lowest, then undoes them and rotates), and prints a readable report. It shows each stage's contents in a canonical (dish) order — the same way the autograder compares, so the report is the **same whatever internal order you keep**. `samples/` pairs a few `.menu/.chosen` files with their `expected_*.txt` outputs, so the `diff` above prints **nothing** when your code is correct (run with the `.chosen` when one exists). Check for leaks with `-fsanitize=address,undefined`.

Grading environment. Your code is compiled and run on Gradescope in **Ubuntu 22.04** with **g++ 11** at `-std=c++20`. Make sure both files compile cleanly there.

HOW IT'S GRADED

Fifty points total, over a memory gate that can zero everything:

- 1. Memory safety first (gate).** Every structure and your node-moving methods run under a memory sanitizer. **Any leak, double-free, or invalid access scores 0** for the whole assignment.
- 2. Your circular list's behavior — 8 pts.** A conformance test builds circular lists by hand and checks `build`/`front`/`back`/`iterate`, `rotate`, `node-detach`, and `node-attach` — four graded groups.
- 3. Your Shortlist's request behavior — 8 pts.** Hand-scripted edge cases: an `add-meal` is rejected when it violates a fixed course, and undoing an `add-meal` destroys the meal; `setCourse` removes the violators and a second `setCourse` on the same course is rejected; `removeLowest` with `k` over the size removes all; and mixed requests undone LIFO across kinds — four graded groups.
- 4. Your methods' results — 18 pts.** For each model the grader grows a shortlist from a clean request script, runs your methods, and compares the resulting list *contents* **order-independently** (it is *which* meals your list holds that is checked, never their order, and never how you represented your undo history).
- 5. Moves, not copies — 8 pts.** A check counts how many Meals your `setCourse` / `removeLowest` / `undoLast` deep-copy. Relinking a node copies none, so a correct transfer scores **0 copies**.
- 6. A large removal and its undo — 4 pts.** The grader runs one *large* `removeLowest`, then one `undoLast`, checks the shortlist is restored exactly, and measures the work each does.
- 7. Performance — 4 pts.** Rewards an efficient organization of your list — *which one is yours to reason out*.

Every correctness test is fully shown to you; some use freshly randomized files. The performance check is graded without spelling out the ideal organization — *that part is yours to reason out*.

Important: Know your own code. You may be **asked about your implementation** in lecture — how the ring's count-bounded cursor works, why a *move* beats a *copy* (and why undoing an `add-meal` is a *destroy*), and how you chose to organize your list — and *why*.

SUBMIT

What to Turn In

Submit on **Gradescope** (the **PA 3,4** assignment). Upload **both** `CircularSinglyLinkedList.hpp` and `Shortlist.hpp` — the filenames must match **exactly**. You may resubmit as often as you like before the deadline; the autograder runs on each submission and shows you the full per-test results.